

Multi-Sensor Pedestrian Detection and 3D Localization

Camera, LiDAR, and Radar Fusion for Autonomous Driving Perception

Tommaso Calzolari

Abstract

This report describes a multi-sensor machine perception pipeline for pedestrian detection and 3D localization in an autonomous-driving scenario. The system uses synchronized camera, LiDAR, and radar measurements to generate candidate pedestrian regions, project them into the camera image, classify the resulting image patches, and return final detections with both image-plane bounding boxes and 3D bird’s-eye-view localization. The implementation follows a modular sensor-fusion structure: LiDAR is used as the main source for geometric proposal generation, radar is used as an additional filtering cue, and the camera is used for appearance-based classification. The final detector produces frame-by-frame pedestrian detections and highlights the main trade-off between recall, precision, and localization robustness in a realistic urban environment.

1 Introduction

Pedestrian detection is a central task in autonomous-driving perception. A vehicle must not only recognize whether a pedestrian is present in the camera image, but also estimate the pedestrian’s spatial position relative to the ego vehicle. This requires combining appearance information from the camera with geometric information from range sensors such as LiDAR and radar.

The objective of this project was to implement a pedestrian detector that operates on measurements from a moving vehicle equipped with a camera, LiDAR, radar, and odometry. The final output of the detector is a set of pedestrian detections for each frame. Each detection contains a class label, a confidence score, the estimated 3D object pose in the camera frame, and the 3D bounding-box extent. These detections can then be projected into the image plane to obtain 2D bounding boxes or visualized in bird’s-eye view (BEV) to evaluate 3D localization.

The project was developed in three stages. First, I inspected the dataset, sensor coordinate frames, calibration matrices, annotations, and ground-plane information. Second, I implemented the core detection pipeline, including point-cloud filtering, clustering, 3D proposal generation, camera projection, patch classification, and non-maximum suppression. Third, I evaluated the detector quantitatively and qualitatively in both the image plane and bird’s-eye view.

2 Dataset and Sensor Setup

The dataset contains synchronized measurements from multiple vehicle-mounted sensors. The relevant modalities for this project are:

- a front-facing camera, used for image-based pedestrian classification;
- a LiDAR sensor, used to obtain 3D point clouds and generate geometric proposals;
- a radar sensor, used to provide additional cues based on reflectivity and radial velocity;
- odometry and precomputed ground-plane information, used for visualization and spatial reasoning.

A key preliminary step was understanding the coordinate frames of the vehicle-mounted sensors and the rigid transformations between them. The detector uses homogeneous transformations to move points between the radar, LiDAR, and camera frames. In particular, radar measurements are transformed into the LiDAR frame before filtering, and LiDAR-based 3D bounding boxes are transformed into the camera frame before projection into the image.

The data-visualization stage also included projecting LiDAR and radar points into the camera image, visualizing annotated objects, creating 3D bounding boxes from object labels, and inspecting the ground plane. These steps were useful for verifying that the calibration and projection pipeline were correct before implementing the detector.

3 Detection Pipeline

The implemented detector follows a proposal-classification architecture. Range sensors are first used to generate candidate 3D pedestrian regions. These proposals are then projected into the camera image, where an appearance-based classifier evaluates whether the corresponding patch is likely to contain a pedestrian.

Figure 1 summarizes the overall structure.

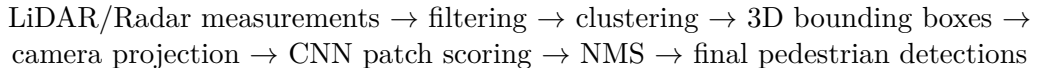


Figure 1: Overview of the multi-sensor pedestrian detection pipeline.

3.1 Radar Filtering

Radar measurements were transformed into the LiDAR frame and filtered using radar cross-section (RCS) and compensated radial velocity. The motivation was to retain measurements that are more likely to correspond to pedestrians while removing reflections from large static structures, vehicles, walls, and other noisy returns.

Two radar masks were used. Moving candidate pedestrian points were accepted with a broader RCS range, while static or near-static candidate points were accepted only under stricter RCS constraints. This reflects the fact that radar points are intrinsically noisy and that a pedestrian may be either moving or almost stationary in a single frame. The filtered radar points were not used as the primary clustering source; instead, they were used to validate and refine the LiDAR-based proposals.

3.2 LiDAR Filtering and Ground-Plane Removal

LiDAR was the main geometric source for proposal generation. The first step was to restrict the point cloud to a region of interest in front of the vehicle and near the roadway. I kept points approximately in the forward range

$$3 \text{ m} < x < 41 \text{ m}$$

and lateral range

$$-15 \text{ m} < y < 15 \text{ m}.$$

The ground plane was then removed using the precomputed ground-plane equation transformed into the LiDAR frame. A distance threshold of approximately 0.15 m was used to remove points close to the plane. This step reduces the number of irrelevant points and helps isolate upright objects such as pedestrians.

3.3 Pillarization and DBSCAN Clustering

After ground-plane removal, I applied a pillarization step to regularize the remaining LiDAR points in the horizontal plane. The point cloud was discretized into small vertical pillars with a pillar size of 0.1 m, keeping only pillars supported by at least two points. This reduced noise and made clustering more stable.

The regularized point representation was then clustered using DBSCAN. The final implementation used a relatively small neighborhood radius and a low minimum-sample requirement, with the goal of separating nearby pedestrians while still retaining sparse clusters at moderate distance. This choice reflects a trade-off: a larger neighborhood radius improves robustness at long range but risks merging nearby pedestrians, while a smaller radius separates close objects but may fragment distant ones.

3.4 3D Bounding-Box Generation

For each LiDAR cluster, I generated an axis-aligned 3D bounding box. The bottom of the box was attached to the estimated ground plane, while the top was determined from the maximum height of the cluster. The candidate boxes were filtered using pedestrian-like geometric constraints, including plausible height and horizontal extent.

The main geometric constraints were:

$$0.5 \text{ m} < h < 2.5 \text{ m},$$

with maximum horizontal extent below approximately 1.8 m. I also required the vertical extent to be larger than the horizontal extent, reflecting the expected shape of a standing pedestrian. For closer detections, radar support and a minimum number of LiDAR points were used as additional validation criteria.

Each retained 3D bounding box was then represented as a pedestrian candidate with a bottom-center position, an extent vector, and a transformation matrix $T_{\text{cam,object}}$ encoding the object pose in the camera frame.

3.5 Projection into the Camera Image

The 3D bounding-box corners were projected into the camera image using the camera projection matrix. From the projected corners, I computed a 2D image-plane bounding box. This 2D box was used to extract an image patch from the camera frame.

This projection step links geometric proposal generation and image-based classification. The 3D proposal defines where a pedestrian might be in space; the camera patch provides visual evidence for or against the proposal.

3.6 CNN-Based Patch Classification

The extracted image patches were evaluated using a CNN-based classifier from the previous practicum. The classifier uses a MobileNetV2-style feature extractor and a neural classification head to produce a pedestrian confidence score for each patch.

The pipeline therefore separates the detection problem into two stages:

1. LiDAR and radar generate candidate regions in 3D.
2. The camera classifier evaluates whether the corresponding image patches visually resemble pedestrians.

This design gave me more control over proposal generation than a single-shot image detector would have. In particular, I could explicitly use LiDAR and radar to reason about 3D location, height, distance, and geometric plausibility before passing candidates to the classifier.

3.7 Non-Maximum Suppression

The classifier scores were post-processed using two suppression steps. First, I applied image-plane non-maximum suppression (NMS) with an IoU threshold of approximately 0.4, removing overlapping detections in the camera view. Second, I applied a distance-based suppression step in 3D, removing detections whose ground-plane centers were too close to a more confident detection.

This combination reduces duplicate detections that arise when multiple clusters or proposals correspond to the same physical pedestrian.

4 Implementation Details

The detector was implemented as a subclass of the provided `PedestrianDetector` base class. For each measurement frame, the detector returns a list of pedestrian dictionaries. Each dictionary contains:

- `label_class`, set to `Pedestrian`;
- `extent_object`, the estimated 3D size of the object;
- `T_cam_object`, the object pose in the camera frame;
- `score`, the classifier confidence.

The detector also supports debug visualizations through the provided debug-output interface. I used these visualizations to inspect LiDAR filtering, radar filtering, DBSCAN clusters, 3D bounding boxes, projected image boxes, extracted patches, and final detections. This was important for iterative development, because many failures in the final detector could be traced back to earlier steps in the pipeline, such as incorrect ground removal, merged clusters, or noisy proposals.

5 Evaluation

The detector was evaluated both quantitatively and qualitatively. The quantitative evaluation considered two complementary views: image-plane projections and bird’s-eye-view localization.

5.1 Image-Plane Evaluation

For image-plane evaluation, the projected 2D bounding boxes of the predicted 3D pedestrians were compared with ground-truth 2D pedestrian boxes. The metrics included precision-recall curves, average precision (AP) at a selected IoU threshold, and mean average precision (mAP) across thresholds.

At an IoU threshold of 0.2, the precision-recall curve showed high precision for low-to-moderate recall values. Precision remained close to one up to roughly 0.4 recall, after which it decreased rapidly. The curve was clipped at about 0.6 recall, meaning that even when the confidence threshold was lowered substantially, the system still missed a fraction of pedestrians.

This behavior has two main causes. First, some pedestrians are occluded in the image, for example by vehicles or other objects. They may be visible to LiDAR or radar, but the projected camera patch does not contain sufficient visual evidence for the classifier. Second, some pedestrians are discarded before classification because the LiDAR cluster is too sparse, too far away, or filtered out during proposal generation. The resulting AP at 0.2 IoU was approximately 50%, while the image-projection mAP was approximately 35%.

5.2 Bird’s-Eye-View Evaluation

Because the task is fundamentally 3D pedestrian localization, image-plane IoU is not the only relevant metric. A detection can be well localized in 3D but still produce an imperfect 2D

box because of projection errors, axis-aligned box assumptions, occlusion, or bounding-box size mismatch.

For this reason, I also evaluated detections in bird’s-eye view using center-distance matching on the ground plane. This metric is closer to the evaluation used in modern 3D detection benchmarks, where the distance between predicted and ground-truth object centers is used as the matching criterion.

The BEV results were better than the image-projection results. This indicates that the fusion pipeline was more effective at estimating lateral and longitudinal pedestrian position than at producing perfectly aligned 2D boxes. In other words, the detector was often able to localize pedestrians in 3D even when the image-plane bounding box was not precise enough to obtain a high IoU.

5.3 Qualitative Evaluation

The qualitative videos revealed several characteristic failure modes.

First, choosing a low confidence threshold improved recall but produced many false positives. In earlier versions, a higher threshold produced cleaner videos and higher precision, but the recall was too low. I therefore favored recall over precision because, in an autonomous-driving context, missing pedestrians is generally more dangerous than producing additional false positives.

Second, detections were sometimes inconsistent frame by frame. This was especially visible in scenes with noisy LiDAR points, overlapping pedestrians, cyclists, or pedestrians walking close to each other. In these cases, the clustering stage could merge multiple pedestrians into one cluster, or the NMS stage could suppress one of two valid nearby detections.

Third, reflections and visually similar structures created false positives. For example, reflections of pedestrians in windows could be classified as real pedestrians if the corresponding region passed the geometric proposal stage.

Fourth, the ground-plane assumption failed in more complex road geometries, such as bridges or non-flat road sections. When the ground plane was not accurately represented by a single plane, ground removal became less reliable and some valid pedestrian points were discarded or incorrectly grouped.

Finally, the BEV visualizations showed that detections tended to appear and disappear across frames. This reflects the lack of temporal tracking: each frame was processed independently, so a detection had no memory from previous frames.

6 Design Assumptions and Limitations

The implementation relies on several simplifying assumptions.

6.1 Planar Ground Model

The detector assumes that the local ground can be approximated by a plane. This assumption is useful for removing ground points and anchoring bounding boxes, but it is not always valid in real urban environments. Slopes, curbs, bridges, and irregular road surfaces can cause the ground-plane estimate to remove useful pedestrian points or leave irrelevant ground points in the cloud.

6.2 Axis-Aligned Bounding Boxes

The generated 3D boxes are axis-aligned. This assumes that pedestrians are oriented consistently with the global axes. In practice, pedestrians may turn, walk diagonally, or appear from the side. Axis-aligned boxes can therefore include unnecessary empty space and project to overly large 2D

image boxes. Oriented bounding boxes would allow the detector to model pedestrian shape more accurately.

6.3 Range-Dependent LiDAR Density

DBSCAN depends on parameters such as the neighborhood radius and the minimum number of samples. These parameters interact strongly with LiDAR point density, which decreases with distance. A pedestrian far from the vehicle may produce too few points to form a valid cluster, while nearby pedestrians may produce dense clusters that are easier to detect. This introduces a range-dependent bias in the proposal stage.

6.4 Sequential Fusion

The pipeline is sequential: each stage filters candidates before passing them to the next. This makes the system interpretable, but also vulnerable to cascading errors. If a true pedestrian is removed during ground filtering, clustering, geometric validation, or radar checking, the later camera classifier never receives the corresponding patch.

7 Future Work

Several improvements would make the system more robust.

First, temporal tracking should be added. The current detector treats each frame independently, which causes flickering detections. A tracking module could increase confidence for objects detected consistently across consecutive frames and suppress isolated false positives. This would improve temporal stability and make the output more usable for downstream planning.

Second, sensor fusion should be made less sequential. Instead of discarding candidates at each stage, a parallel or probabilistic fusion approach could combine LiDAR, radar, and camera evidence simultaneously. This would reduce cascading errors and allow weak evidence from one modality to be compensated by stronger evidence from another.

Third, the geometric model could be improved. Adaptive ground-plane estimation and oriented 3D bounding boxes would address several observed limitations, including sloped roads, bridges, curbs, and pedestrians walking at different orientations. These changes would likely improve both 3D localization and 2D projection accuracy.

8 Conclusion

This project implemented a complete multi-sensor pedestrian detection pipeline for an autonomous-driving perception task. The detector combines LiDAR-based 3D proposal generation, radar-assisted filtering, camera-based CNN patch classification, and non-maximum suppression. The resulting system produces frame-by-frame pedestrian detections with both 2D image-plane bounding boxes and 3D BEV localization.

The main strength of the approach is that it explicitly uses the complementary properties of the three sensors: LiDAR provides geometry, radar provides motion and reflectivity cues, and the camera provides semantic visual information. The main limitation is that the fusion is sequential and relies on simplifying geometric assumptions, especially a planar ground model and axis-aligned bounding boxes.

Overall, the project demonstrates a full perception pipeline for pedestrian detection and 3D localization, including dataset visualization, coordinate transformations, point-cloud processing, proposal generation, CNN-based classification, quantitative evaluation, qualitative video analysis, and reflection on real-world deployment limitations.